

AEGIS GUARD

Behavioral Twin Prompt Injection Firewall

ET AI Hackathon: Agentic Edition · Problem 2 · Solo

Self-estimated 9-blocker: F3 / D2

Live · aegis-prompt-injection-firewall.vercel.app

The enterprise risk

Agents touch untrusted content

PDF, email, HTML, OCR, and APIs now feed tool-using agents. Attackers hide instructions inside those documents.

Indirect prompt injection

A single poisoned attachment can trigger `get_secret`, `send_email`, or shell tools — without the user noticing.

Classifiers are not enough

Keyword / LLM jailbreak detectors ask: “Does this look bad?” Attackers evade that every day.

Accenture clients are shipping copilots and agents into production workflows.

Without a runtime control plane, every document becomes an attack surface.

The insight: runtime, not classification

Prompt injection is not a text-classification problem.

It is a runtime security problem:

Does untrusted content change what tools the agent would call?

Aegis Guard runs a behavioral twin:

- Trusted plan — instructions with untrusted content stripped
- Raw plan — instructions as attackers hope the agent will see them
- If plans diverge on high-risk tools → BLOCK or QUARANTINE

Everyone else ships a classifier. We ship a twin.

Aegis Guard control plane

01 Ingest

Normalize PDF, HTML, email, OCR, API

02 Decode

Unwrap base64 / hex / rot13 / homoglyphs

03 Provenance

Tag trusted vs untrusted spans

04 Static

Detect 9 attack families

05 Twin

Dual tool-plan comparison

06 Risk + Gate

ALLOW · QUARANTINE · BLOCK

Architecture — evidence for claims

Ingest → Decode → Provenance → Static → Behavioral Twin → Risk → Gate

Protected Agent (capability tokens) · Analyst Quarantine (HITL)

- Capability tokens fail-closed on secrets, shell, external email
- SQLite audit trail + incident JSON export
- Session risk memory for multi-step jailbreaks
- Optional OpenAI-compatible LLM planner (offline twin by default)
- Stack: Python · FastAPI · Streamlit (local) · Vercel production UI
- Repo: github.com/krabhi75/aegis-prompt-injection-firewall

Demo proof

Benign product PDF	ALLOW	Trusted summarize path
Indirect injection in PDF	BLOCK	Twin drift: get_secret + send_email
Encoded HTML jailbreak	BLOCK	Decode layer + override
Multi-step roleplay → tools	QUARANTINE / BLOCK	Session risk escalation
Human analyst queue	HITL	Approve / deny with audit note

9-blocker claim — F3 / D2

F3 — Features (≥ 7 attack types)

All 9 official attack types implemented and covered in corpus:

- 1 Instruction Override
- 2 Role Change
- 3 Secret Extraction
- 4 Tool Abuse
- 5 Credential Theft
- 6 Context Poisoning
- 7 Multi-Step Jailbreak
- 8 Encoded Instructions
- 9 Indirect Prompt Injection

Live demo demonstrates ≥ 7 clearly.

D2 — Depth (high reliability)

Mostly structured/textual multimodal inputs (user, PDF text, HTML, email, API, OCR text) with demonstrable reliability:

- Corpus $n = 55$
- Precision / Recall / F1 ≈ 1.0 on suite
- Pytest: 11/11 passed
- Command Center metrics in live UI

Not claiming D3: Word/image OCR not fully shown in production demo.

Business impact & ask

Measurable outcomes for enterprises deploying agents:

- Risk reduction — block secret theft and tool abuse before execution
- Auditability — incident JSON + decision trail for security teams
- Governance — OWASP LLM Top 10 & NIST AI RMF mapped controls
- Human oversight — quarantine band keeps humans in the loop

Live: <https://aegis-prompt-injection-firewall.vercel.app>

Code: <https://github.com/krabhi75/aegis-prompt-injection-firewall>

Closer: Everyone else ships a classifier. We ship a twin.

If the document tries to make your agent steal secrets,

the plans diverge — and Aegis blocks before a single tool runs.